Here are some best practices for creating custom Hooks:

- *Single responsibility principle*—Keep hooks focused on a single responsibility to maintain simplicity and reusability.
- *Avoid complexity*—Refrain from overcomplicating hooks with too much logic. Instead, break down complex logic into smaller, more manageable hooks.
- *Meaningful names*—Use meaningful names that describe the hook's purpose, making it easier for others to understand the hook's functionality.

Custom Hooks are frequently discussed in React interviews, especially for roles involving advanced state management and reusable component design.

A.4.3 The use API

React 19 introduces a new API, `use` (https://react.dev/reference/react/use), for reading resources during rendering. These resources can be either:

- *Promises*—Handle asynchronous operations directly in the render phase, allowing components to suspend until data is ready.
- *Contexts*—Access shared data without explicitly using `useContext`.

Unlike Hooks, `use` can be called conditionally during rendering, offering more flexibility.

The syntax for using this API is:

```
const value = use(resource);
```

- The `use` API accepts a single parameter, `resource`, which serves as the source of the data to be read. A resource can be either a Promise or a Context. The example is located at
- The API returns a `value`, which is the data read from the resource—whether it originates from a Promise or a Context.

The example of `use(context)` is located in `use-cases/RenderComponentExample.tsx`, and it can be executed with the